

Disaster Recovery Plan — Van Reservation System (VRS)

Database Platform: Company-Managed PostgreSQL (Current)

Organization	Carelon Global Solutions PH
Prepared by	Arielle Jimera
Date	2026-09-18
Status	Draft v0.1 — Pending Review
Classification	Internal — Confidential

This document defines disaster recovery procedures for a single database platform variant of the Van Reservation System. A companion plan exists for the other platform variant — confirm which is in effect for the live production environment before acting on it during an incident.

1. Purpose

This document defines the Disaster Recovery (DR) Plan for the Van Reservation System (VRS), an internal Next.js/PostgreSQL application used by Cargon Global Solutions PH to book, approve, and manage company van transportation across the Manila and Iloilo sites. It replaces the manual/Nintex-based process that preceded it, so an extended outage has no automatic manual fallback.

This plan covers the database platform as currently designed for deployment (company-managed PostgreSQL). A companion document covers the target-state Amazon RDS for PostgreSQL platform; the two are intentionally separate because the recovery mechanics, achievable RPO, and required infrastructure differ materially between them.

The plan exists to ensure that, in the event of a disruptive incident, the system and the people responsible for it know: what must be recovered, in what order, by whom, how success is verified, and how the incident is closed out and reported.

2. Objectives

The primary objective of this plan is to restore the Van Reservation System (VRS) to full production functionality within the Recovery Time Objective (RTO) and Recovery Point Objective (RPO) below, and to confirm — before and during any recovery — that the supporting information in this document (asset scope, contacts, recovery tasks, and the application diagram) is current and accurate.

Metric	Target	Definition
RTO (Recovery Time Objective)	4 hours	Maximum acceptable time from disaster declaration to full production functionality restored for all roles (requestor, admin) and all pages.
RPO (Recovery Point Objective)	1 hour	Maximum acceptable data loss, measured as the age of the most recent recoverable transaction relative to the moment of failure.

Asset	RTO contribution	RPO	Notes
Application (Next.js)	~45 min	N/A — stateless	Rebuilt/redeployed from source control; no state to lose.
Database (PostgreSQL)	~2.5 hours	1 hour	Manual/scripted restore from the most recent base backup plus WAL replay. This is the long pole in the budget and is NOT currently met by a default `pg_dump`-only backup strategy — see the gap noted in §6.
Email / notification_outbox	Resumes with app	0 — queue is durable	Queued rows survive a database restore; the outbox sweeper drains the backlog after recovery.

Asset	RTO contribution	RPO	Notes
CGS Auth API integration	Not recoverable by this team	N/A — external	Dependent on the CGS platform team's own availability; this plan cannot recover it.

Note. Gap to close before this RTO/RPO is achievable in practice: a scheduled, tested backup regime (continuous WAL archiving or an equivalent tool such as pgBackRest/Barman, or a managed-backup offering from the hosting provider) is not yet in place for the company-managed PostgreSQL server, per operations.md's production readiness checklist ("Managed Postgres provisioned, backups configured" is unchecked as of this writing). Until that is provisioned, RPO is bounded only by the interval between manual backups, which may be far worse than 1 hour.

3. Scope

This plan covers the Van Reservation System (VRS) application tier, its database, its two external digital integrations, and the servers/compute that host them. It does not cover the CGS Associate Authentication API's own disaster recovery (owned by the CGS platform team), the corporate network or Active Directory that identity ultimately rests on, end-user devices, or the legacy Nintex-based process VRS replaced (deprecated, not maintained as a fallback).

3.1 Asset Scope

Applications

Asset	Description
van-reservation (web app)	Next.js 16 App Router application, TypeScript/React 19. 13 pages across three route groups: (admin) — dashboard, master-list, calendar, reports, drivers, audit-logs, roster; (auth) — login, admin/login, not-authorized; (requestor) — home, book, manage.
API routes	Route handlers under /api: auth (login/logout/session/verify-domain), associates/lookup, reservations (+ [id], + cancel), drivers, vans, admins (+ [id], + affected), audit-logs, and the internal outbox sweeper at /api/internal/dispatch-outbox.
Build/runtime	Node.js 20+, pnpm 9+. Production runs `pnpm build` then `pnpm start`. No `engines` field is pinned in package.json — version drift is a known gap, not enforced by tooling.

Databases

Asset	Description
Primary database	PostgreSQL 17 (pinned as postgres:17-alpine in the reference Docker Compose; 13+ supported), company-managed server/VM. Connection via a single `pg` Pool capped at 10 connections.

Asset	Description
Schema	11 tables via 13 ordered Kysely migrations: users, admin_whitelist, drivers, vans, reservations, reservation_passengers, reservation_events (append-only), roster_events (append-only), notification_events, notification_outbox, reference_counters.
Schema type source of truth	`kysely-codegen` generates src/modules/db/types.d.ts from the live schema — never hand-edited. Must be regenerated after any migration.
Backups	Not yet provisioned as of this writing (see \$2 gap note). Must be established before this system carries production data.

Digital Integrations

Integration	Description	DR ownership
CGS Associate Authentication API	External identity provider. `identify`, `verifyDomain`, `lookupName` through the AuthProvider interface (CgsAuthProvider in production, AUTH_MODE=cgsauth). No local password store — a successful `identify` is the admission check.	External — owned by the CGS platform team, not this plan.
AWS SES v2 (email)	Outbound notification delivery. EMAIL_MODE=ses in production; templates rendered with React Email. Requires a verified, DKIM-signed sender identity out of the SES sandbox.	Shared — AWS-side availability is AWS's; sender identity, credentials and dispatch scheduling are this team's.
Outbox sweeper	GET /api/internal/dispatch-outbox, called on a schedule (recommended every 5–15 minutes) guarded by DISPATCH_SECRET. The only drain path that runs when nobody is actively booking a trip.	Internal — this team owns the scheduler and the secret.

Servers

Component	Description
Application host	Node.js process/container serving `pnpm start`, listening internally on port 3000 behind a reverse proxy. Exact host, OS image, and scaling model: [TBD — Infrastructure Team to confirm for the production deployment].
Database host	Company-managed PostgreSQL server/VM. Exact host, capacity, storage/IOPS provisioning, and network segment: [TBD — Infrastructure Team to confirm].
Reverse proxy / load balancer	[TBD — Infrastructure Team to confirm the production topology; not present in the local Docker Compose reference, which exposes only the database on host port 5433].
Scheduler for the outbox sweeper	[TBD — cron, a task scheduler, or a cloud scheduled job; must call the sweeper every 5–15 minutes with a valid DISPATCH_SECRET header].

4. Roles and Responsibilities

DR execution follows a small, named chain of ownership rather than being everyone's job. The roles below are organizational roles, not VRS's own application roles (associate / admin_support) — do not confuse the two.

Role	Responsibility during a DR event
DR Plan Owner / IT Manager	Declares the disaster, activates this plan, owns the go/no-go on standing down, and is the single point of authority for the duration of the incident.
Application/Dev Lead	Directs application-tier recovery: redeploys the app, verifies environment configuration, and runs the functional smoke test in §6.
Database Administrator (DBA) / Infra-Cloud Engineer	Directs database recovery: restore, failover, or PITR as appropriate to the incident; verifies schema and data integrity before handing back to the App Lead.
Security / Compliance Officer	Assesses whether the incident involves a security or data-privacy dimension (e.g. suspected breach) and triggers any required regulatory notification track in parallel with technical recovery.
Communications Lead	Owns status updates to stakeholders and end users per §8, so technical responders are not interrupted to answer “any update?”.
Site Admin Coordinators (Manila, Iloilo)	Confirm, once the app is back, that site-specific data (fleet roster, pending approvals) looks correct from the admin side, and relay status to their site's requestors. Mirrors the existing site-based admin notification routing (admin_whitelist.site / notify).

4.1 Contacts

This table is a live document — confirm it is current at the start of every DR test and after every personnel change. Names, emails and phone numbers below are placeholders pending confirmation by the IT Manager; do not treat blank cells as “no one owns this.”

Role	Name	Email	Phone	Escalation order
DR Plan Owner / IT Manager	[TBD]	[TBD]	[TBD]	1
Application/Dev Lead	[TBD]	[TBD]	[TBD]	2
Database Administrator	[TBD]	[TBD]	[TBD]	2
Infrastructure/Cloud Engineer	[TBD]	[TBD]	[TBD]	3
Security/Compliance Officer	[TBD]	[TBD]	[TBD]	3
Communications Lead	[TBD]	[TBD]	[TBD]	3

Role	Name	Email	Phone	Escalation order
Site Admin Coordinator — Manila	[TBD]	[TBD]	[TBD]	4
Site Admin Coordinator — Iloilo	[TBD]	[TBD]	[TBD]	4
CGS Auth Platform Team (external)	[TBD]	[TBD]	[TBD]	on integration failure only

5. Activation Criteria

This plan is activated by the DR Plan Owner when one or more of the following is true. Activation is a formal act — it starts the clock on the RTO and triggers the notification chain in §4.1.

- The application is fully unreachable, or every request 500s, for longer than 15 consecutive minutes.
- The PostgreSQL server is unreachable, refuses connections, or its data is confirmed corrupted/lost.
- The company data center or the host running PostgreSQL/the application suffers a facility-level outage (power, network, hardware failure) with no defined recovery time from Infrastructure.
- A failed deployment or migration has broken production and cannot be rolled back within 30 minutes (see the migration-6 hazard in §6).
- A security incident (ransomware, unauthorized access, data exfiltration) has compromised the application or database host.
- Data corruption is discovered that affects the integrity of reservations, the audit trail (reservation_events/roster_events), or the admin whitelist.

Note. Severity levels: SEV1 — full outage or confirmed data loss/corruption, this plan activates immediately. SEV2 — degraded but partially functional (e.g. email delivery down but booking works), the DR Plan Owner decides whether to activate based on duration and business impact. A SEV2 that is not resolved within 2 hours escalates to SEV1 by default.

6. Actions to Implement

Recovery proceeds in the phases below. Each phase names its owner from §4. Phases 3 onward differ meaningfully between the two database platforms and are written for this plan's platform only.

Phase 0 — Detection & Declaration (Owner: DR Plan Owner)

- 1 Confirm the incident against the Activation Criteria in §5 — do not activate on a single unconfirmed alert.
- 2 Formally declare the disaster and record the declaration time; this is T+0 for RTO tracking.
- 3 Open an incident log (timestamp / action / owner / outcome) — required input to the post-incident report in §8.

Phase 1 — Notification (Owner: DR Plan Owner / Communications Lead)

- 1 Notify all contacts in §4.1 in escalation order.
- 2 Post an initial status to the agreed stakeholder channel [TBD — name the channel/distribution list], including expected next update time.

Phase 2 — Assessment (Owner: App Lead + DBA/Infra jointly)

- 1 Determine the failure domain: application, database, network, or an external integration (CGS Auth, SES).
- 2 For an integration-only failure, confirm the application and database are otherwise healthy before treating this as a full DR event — recovery for CGS Auth outages is out of this plan's control (§9) and mainly a communications problem, not a technical one.
- 3 Decide the recovery path for the database (Phase 3) based on the failure type: infrastructure failure vs. data corruption/accidental loss require different actions.

Phase 3 — Database Recovery (Owner: DBA)

- 1 If the server itself failed: provision/repair a replacement PostgreSQL 17 host per the specification in §3.1.
- 2 Restore the most recent base backup, then replay WAL/transaction logs (if archiving is in place) to minimize data loss toward the 1-hour RPO. If only periodic dumps exist, restore the latest dump and record the actual data-loss window for the post-incident report — do not overstate RPO achieved.
- 3 Run ``pnpm db:migrate`` only if the restored database predates a migration that has since shipped; otherwise skip — restoring a current backup already carries the current schema.
- 4 Verify row counts and referential integrity on the core tables: `reservations`, `reservation_passengers`, `admin_whitelist`, `drivers`, `vans`.
- 5 Verify `reference_counters` is intact and its per-year sequence has not reset — a reset risks duplicate `VR-YYYY-#####` reference numbers on the next booking.
- 6 Verify `notification_outbox` for rows stuck in ``sending`` (indicates a process was killed mid-restore); these self-heal via the retry/backoff described in `operations.md`, but confirm none are orphaned past `MAX_ATTEMPTS`.
- 7 Do NOT re-run ``pnpm db:seed`` reflexively to “fix” missing data — it re-asserts `admin_whitelist` and `fleet` fields and will revert any legitimate post-seed edits an admin made via `/roster` (active/deactivate status is the only field the seed never touches). Only run it if the whitelist itself is confirmed missing or corrupted.
- 8 Hand back to the App Lead once the database accepts connections and passes the checks above.

Phase 4 — Application Recovery (Owner: App Lead)

- 1 Provision or restart the application compute per §3.1.
- 2 Verify required environment variables are present and correct: `SESSION_SECRET` (32+ chars), `AUTH_MODE=cgsauth` with a valid `CGS_AUTH_API_KEY`, `EMAIL_MODE=ses` with `AWS_REGION` and `EMAIL_FROM`, the `DATABASE_URL/DB_*` pointing at the recovered database, and `DISPATCH_SECRET`.
- 3 Deploy the last known-good build: ``pnpm install``, ``pnpm build``, ``pnpm start`` (or restart the existing container/service if the compute itself was not lost).
- 4 Confirm the app is serving: the login page renders and ``GET /api/auth/session`` responds.

Phase 5 — Integration Verification (Owner: App Lead)

- 1 Confirm CGS Auth reachability with a real sign-in attempt at /admin/login and /login.
- 2 Confirm SES is actually sending, not just queuing: submit a test booking and confirm the corresponding notification_outbox row reaches status='sent' with a provider_message_id set — status='sent' alone can also mean the stub, so also confirm EMAIL_MODE=ses is set in the running environment.
- 3 Confirm the outbox sweeper's scheduler has resumed calling GET /api/internal/dispatch-outbox with the correct DISPATCH_SECRET header, and that it returns 200 with a {sent, failed, retrying} summary rather than 404 (misconfigured secret) or 401 (wrong secret).

Phase 6 — Data Integrity Verification (Owner: DBA + App Lead)

- 1 Spot-check total reservation counts against the last known-good figure (from monitoring or the last pre-incident report) to bound any silent data loss.
- 2 Scan reservation_events and roster_events for chronological gaps around the incident window — both are append-only, so a gap is a strong corruption signal.
- 3 Confirm the unique constraints that guard identity and fleet data are intact: lower(email) and lower(domain_id) on users, lower(trim(email)) on admin_whitelist, and plate on vans.
- 4 Confirm admin_whitelist.super_admin holders match the expected roster — if db:seed had to be re-run, re-verify who holds it rather than assuming it matches pre-incident state.

Phase 7 — Functional Smoke Test (Owner: App Lead, with a Site Admin Coordinator)

- 1 Sign in as a requestor and submit a test trip through the booking wizard.
- 2 Sign in as an admin, locate the test trip on /master-list, and approve it, assigning a driver and van.
- 3 Confirm the requestor received (or, in stub mode only, would have received) the approval notification, and that the trip appears correctly on /calendar.
- 4 Confirm the action appears on /audit-logs.
- 5 Cancel the test trip to leave no residue in production data.

Phase 8 — Cutover / Go-Live (Owner: DR Plan Owner)

- 1 If the application's public endpoint (APP_URL / DNS) changed as part of recovery, update it and allow for propagation before declaring full availability.
- 2 Declare the system restored and move to §7, Process for Standing Down.

6.1 Recovery Tasks — Time Budget (within the 4-hour RTO)

Phase	Task	Target duration	Cumulative
0–1	Detection, declaration, notification	30 min	0:30

Phase	Task	Target duration	Cumulative
2	Assessment / failure-domain triage	20 min	0:50
3	Database restore (base backup + WAL replay, or latest dump)	up to 2:00	2:50
4	Application redeploy/restart and config verification	30 min	3:20
5	Integration verification (Auth, SES, sweeper)	20 min	3:40
6–7	Data integrity check and functional smoke test	20 min	4:00

Note. This budget assumes a backup strategy capable of restoring within ~2 hours is in place. Without one (see the §2 gap note), Phase 3 is the task most likely to overrun the 4-hour target, and this should be flagged to the DR Plan Owner in real time rather than silently absorbed.

7. Process for Standing Down

Standing down closes the incident. It is a deliberate decision by the DR Plan Owner, not an automatic consequence of the smoke test passing.

- 1 Confirm every check in Phase 7 (Functional Smoke Test) passed.
- 2 Hold a soak period of at least 2 hours with normal monitoring in place and no recurrence of the triggering condition before declaring full recovery.
- 3 Turn off any temporary compensating measures put in place during the incident (e.g. a manual email fallback, a maintenance banner, throttled access).
- 4 Notify all §4.1 contacts and the stakeholder channel that the system is fully restored, including the final RTO/RPO actually achieved.
- 5 Formally close the incident log opened in Phase 0, with an end timestamp.
- 6 Schedule a post-incident review within 5 business days (see §8).
- 7 File this plan's corrections — anything that did not match reality during execution (wrong contact, missing step, wrong duration) gets fixed in this document before the next drill, not left for the next real incident to rediscover.

8. Reporting Requirements

During the incident

- Status updates to the stakeholder channel [TBD — name the channel] every 30 minutes during a SEV1, every 60 minutes during a SEV2.

- Each update states: current phase (per §6), time elapsed against the 4-hour RTO, and the next expected update time.
- The incident log (timestamp / action / owner / outcome) is maintained continuously and is the primary source for the post-incident report.

After the incident

- A Post-Incident Report is produced within 5 business days of standing down, owned by the DR Plan Owner.
- Contents: full timeline, root cause, RTO/RPO targeted vs. actually achieved, any confirmed data loss (with the affected reservation/audit reference numbers if applicable), corrective actions with owners and due dates, and sign-off from the DR Plan Owner and the Application Owner.
- If the incident had a security or data-privacy dimension, the Security/Compliance Officer coordinates any required regulatory notification on its own timeline, in parallel with this report — do not wait for the technical report to start that track.
- Reports are retained per company records-retention policy [TBD — confirm retention period with Compliance].

9. Internal–External Dependencies

Internal

Dependency	Why it matters to recovery
Corporate network / VPN access	Responders need connectivity to the hosting environment and to each other's tools during recovery.
Source control (the VRS git repository)	The application is rebuilt from source on redeploy; the repository's availability is a hard dependency for Phase 4.
Package registry (npm/pnpm registry)	`pnpm install` during redeploy needs registry access; an outage there blocks Phase 4 independently of this system's own infrastructure.
Infrastructure/Ops team	Owns the physical or virtual hosts, network, and (for the PostgreSQL plan) backup tooling this plan assumes exists.

External

Dependency	Why it matters to recovery	Controlled by
CGS Associate Authentication API	No one can sign in — as requestor or admin — while this is down. This plan cannot recover it; recovery is limited to detecting and communicating the outage.	CGS platform team
AWS SES	Outbound notification delivery. An SES-side outage stalls the outbox but does not block booking/approval, which do not wait on email synchronously.	AWS

Dependency	Why it matters to recovery	Controlled by
Company data center / hosting provider	Owns the physical facility, power, and network the application and database servers run in.	Company Infrastructure team / facility provider
DNS provider	Needed if APP_URL/DNS must be reprinted during recovery.	[TBD — name the provider]

Note. Any dependency this team does not control (CGS Auth, AWS platform health) is explicitly out of this plan's ability to recover — the plan's job for those is detection, communication, and waiting, not remediation.

Appendix A: Application Diagram

Requestor/Admin Browser → Next.js App (App Router; edge middleware gates admin pages) → API routes → domain modules (auth, reservations, admins, drivers, vans, roster, audit, email) → Kysely → PostgreSQL (company-managed server). Two external dependencies sit beside the app: the CGS Associate Authentication API (identity) and AWS SES (outbound email). A scheduler calls the internal outbox sweeper endpoint on an interval to drain notification_outbox independently of live traffic.

